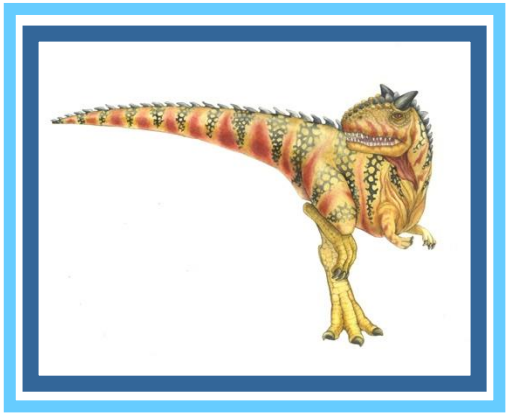
Chapter 9: Main Memory



Silberschatz, Galvin and Gagne ©2018



Chapter 9: Main Memory

- Logical vs. physical memory
- Program transformations, relocation and address binding
- Contiguous allocation and free-space management
- Fragmentation, the 50% rule, swapping and compaction
- Dynamic linking and sharing
- Paging
- Segmentation, and segmentation with paging
- Translation look-aside buffer (TLB)





Objectives

- Explain how a program's logical addresses are bound to physical addresses (static and dynamic relocation)
- Compare first-fit, next-fit, best-fit and worst-fit, and explain external fragmentation and the 50% rule
- Translate logical addresses to physical addresses under paging, segmentation, and segmentation with paging
- Compute address sizes, internal fragmentation, and the effective access time with a TLB





Background

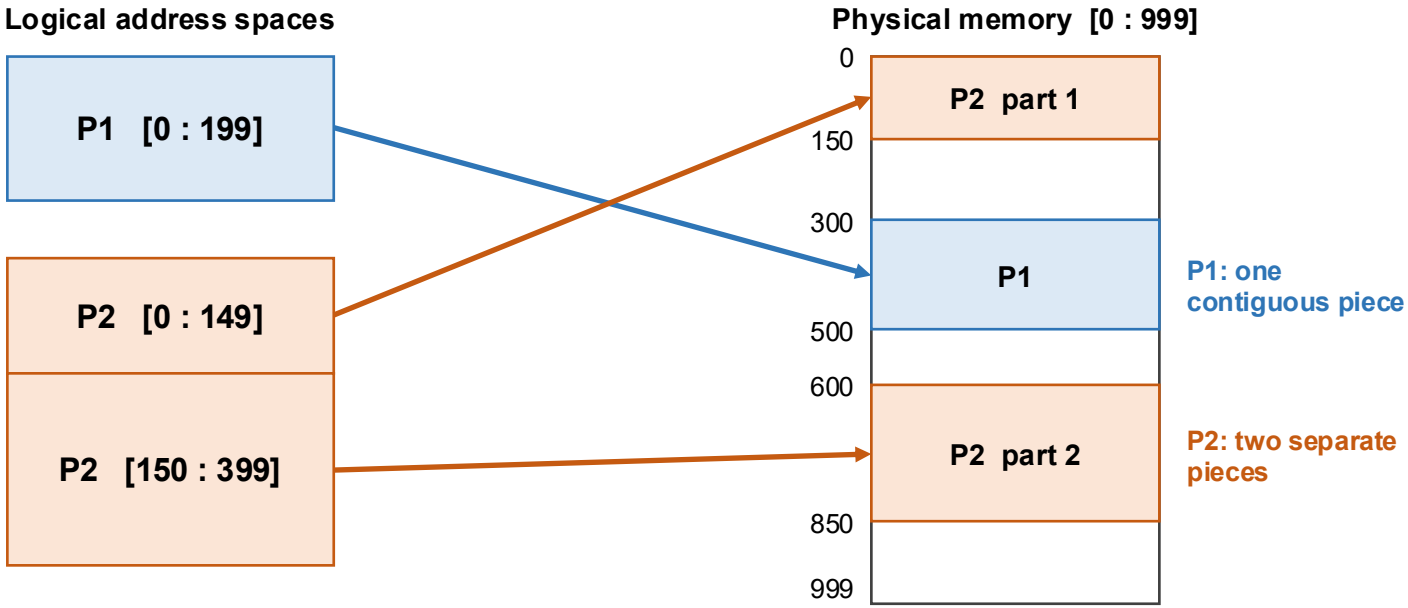
- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of:
  - addresses + read requests, or
  - address + data and write requests
- Register access is done in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation





Logical vs. Physical Memory

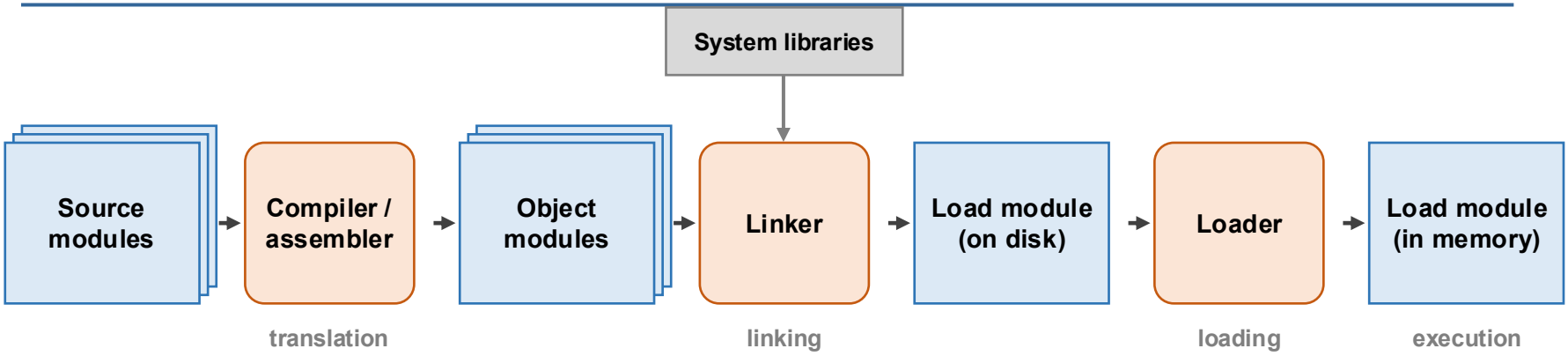
- **Physical memory** (RAM): n words with **physical addresses** [0 : n-1]
- **Logical address space**: an imaginary memory [0 : m-1]; every program is written as if it starts at address 0
- Before execution, the logical space is **mapped** to part of physical memory, in one piece or in several pieces
- A memory of n words needs a k-bit address, where $n = 2^k$ (4096 words need 12-bit addresses)







Program Transformations



- **Source module**: a program part written in C or assembly language
- **Object module**: machine code made from one source module by the compiler or assembler
- **Load module**: object modules (and libraries) combined by the **linker**, ready to run
- The **loader** copies the load module from disk into memory before execution

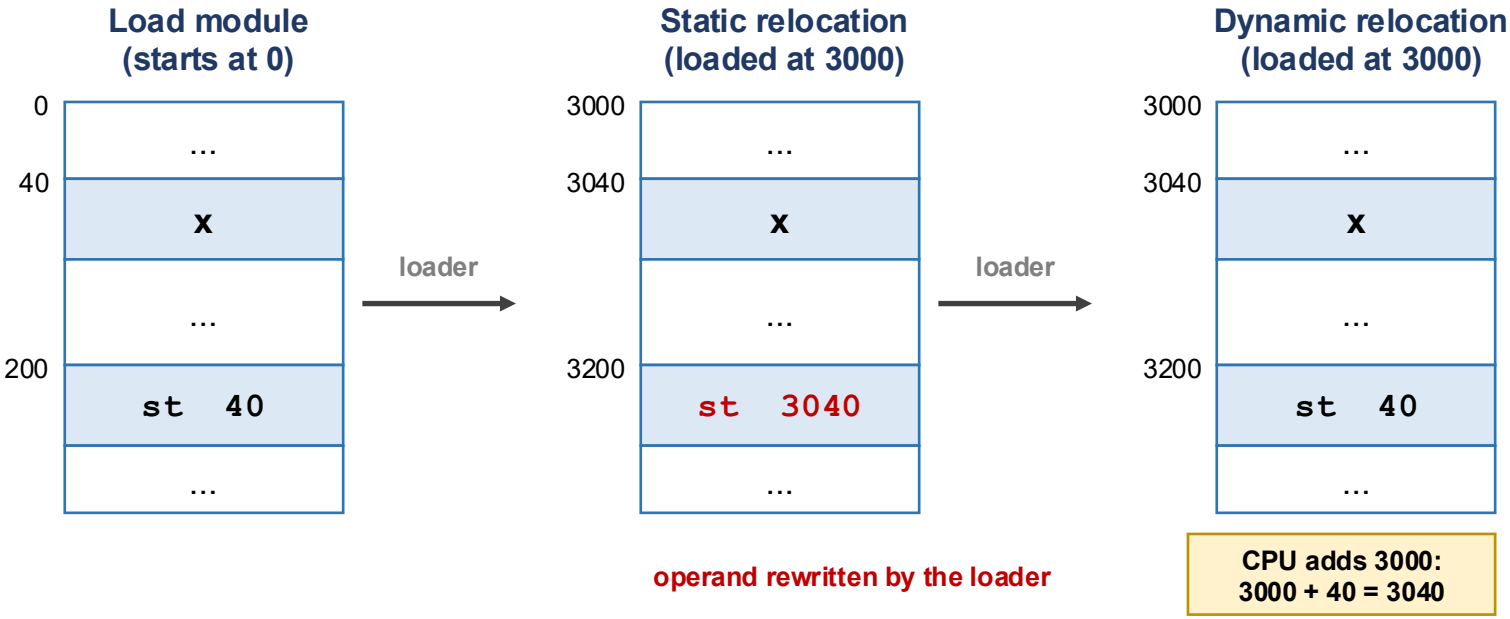
The input to the linker is **object modules**; the input to the loader is **a load module**.





Relocation: Static vs. Dynamic

- Compiler and linker assume every module starts at **address 0**; the real start is known only at load time
- **Relocation**: moving a program from one address space to another
- **Static relocation**: the loader rewrites every address **before** execution
- **Dynamic relocation**: addresses stay logical; the CPU adds the start address **at run time**, on every access





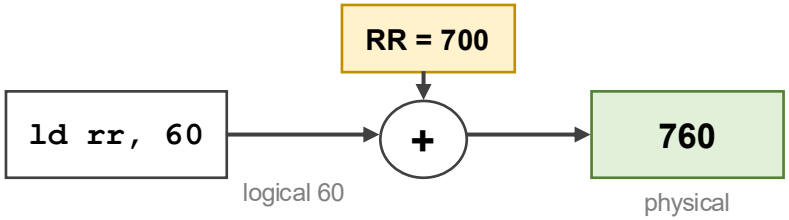


Relocation Registers

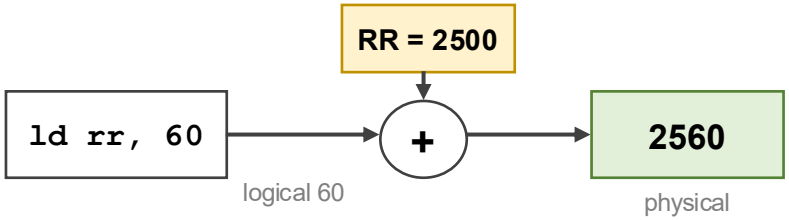
- A **relocation register** holds the physical start address of a program (or one part of it)
- The CPU adds it to every logical address; moving the program only needs a new register value

One register (RR) for the whole program

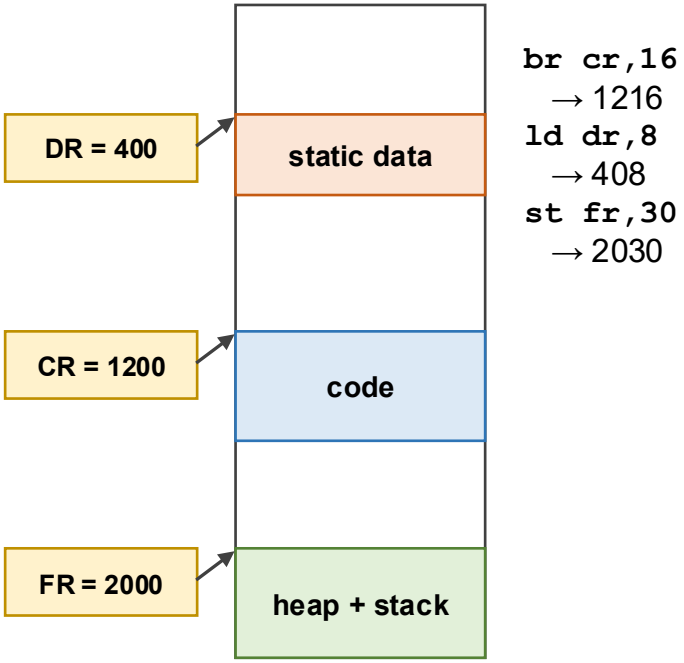
Program loaded at 700



Program moved to 2500: only RR changes



Three registers: code, static data, dynamic data







Practice: Relocation

- A load module keeps variable y at logical address 25. It is loaded into memory starting at address 1800.
  - (a) At which physical address does y reside?
  - (b) With static relocation, what does the instruction `ld 25` become?
  - (c) With dynamic relocation, what does it become, and what is in RR?
- A program has separate code, static data and dynamic data areas. Which register (CR, DR, FR) does each use?
  - (d) a branch inside the code (e) loading a global variable (f) pushing onto the stack

Answers

(a) 1800 + 25 = 1825

(b) `ld 1825`: the loader rewrote the operand

(c) `ld 25` is unchanged; RR = 1800, so the CPU forms 1825 at run time

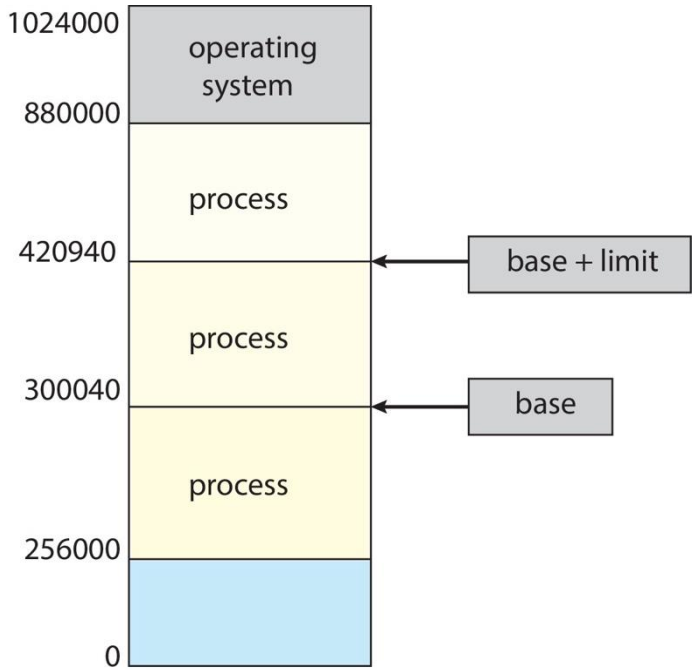
(d) CR (e) DR (f) FR





Protection

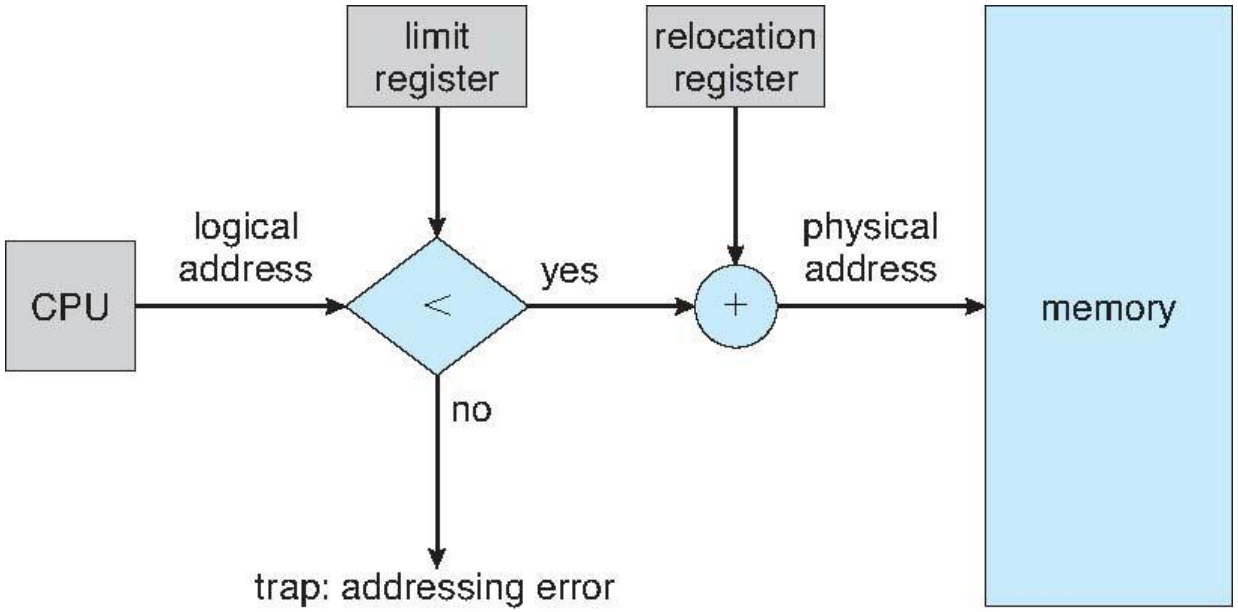
- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** define the logical address space of a process







Hardware Support for Relocation and Limit Registers



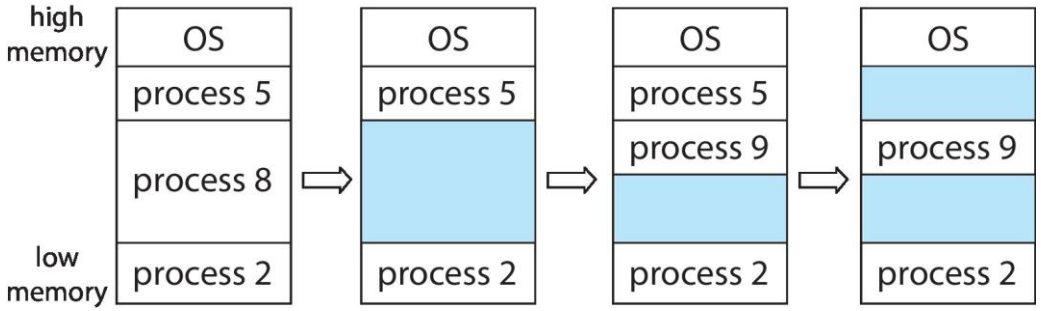
Example: relocation = 4000, limit = 900. Logical 350 → 4350. Logical 950 ≥ 900 → trap





Variable Partition

- Multiple-partition allocation
  - Degree of multiprogramming limited by number of partitions
  - **Variable-partition** sizes for efficiency (sized to a given process' needs)
  - **Hole** – block of available memory; holes of various size are scattered throughout memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Process exiting frees its partition, adjacent free partitions combined
  - Operating system maintains information about:
    a) allocated partitions b) free partitions (hole)

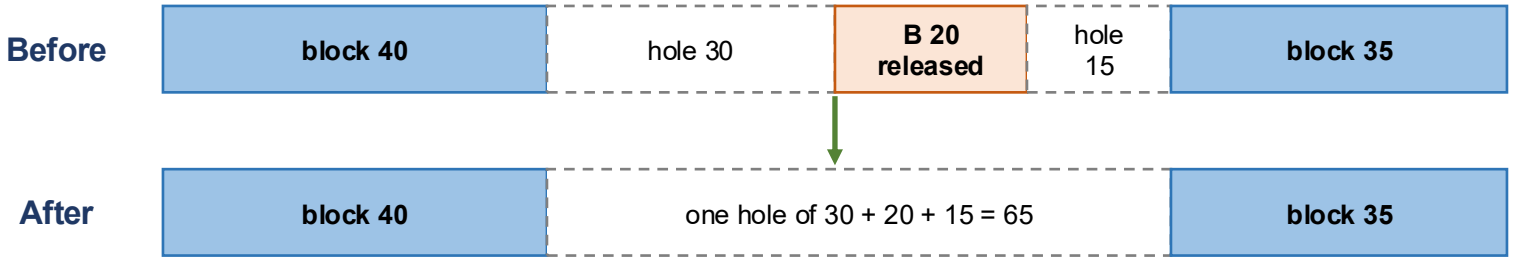






Free-Space Management

- Programs come and go, so memory becomes a checkerboard of blocks and **holes** (free areas)
- The OS keeps the holes in a linked list and chooses one for each request:
  - **First-fit**: the first hole that is large enough (each search starts at the beginning)
  - **Next-fit**: like first-fit, but starts where the last search stopped
  - **Best-fit**: the smallest hole that is large enough (searches all holes)
  - **Worst-fit**: the largest hole (searches all holes)
- **Coalescing**: a released block is merged with the holes next to it







Placement: Worked Example

- Holes in memory order: H1 = 700, H2 = 550, H3 = 100, H4 = 300, H5 = 720
- Requests arrive in this order: A = 80, B = 390, C = 650, D = 540

	H1	H2	H3	H4	H5	Result
Before	700	550	100	300	720	
First-fit	A, B (230)	D (10)	(100)	(300)	C (70)	all placed
Next-fit	A, D (80)	B (160)	(100)	(300)	C (70)	all placed
Best-fit	C (50)	B (160)	A (20)	(300)	D (180)	all placed
Worst-fit	B (310)	(550)	(100)	(300)	A, D (100)	**C fails**

Number in parentheses = what is left of the hole

- Worst-fit used up the big hole H5 early, so C (650) found no hole: worst-fit breaks up large holes
- Best-fit left a 20-word hole in H3 that is too small to use

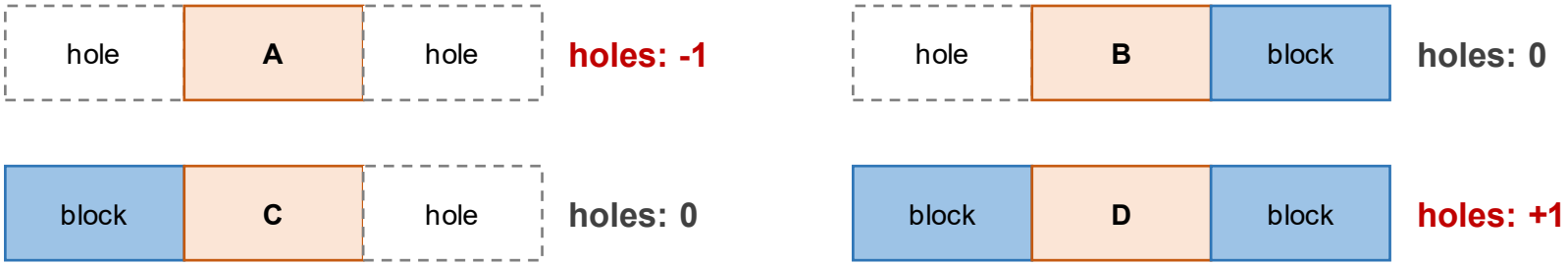




Fragmentation and the 50% Rule

- **External fragmentation**: free space is split into holes between blocks; the total may be enough, but no single hole is
- **50% rule**: in a steady state, #holes = 0.5 × #blocks, so **1/3 of all partitions are holes**. No placement strategy can change this
- If a hole is as large as a block on average, **1/3 of memory is wasted**
- With k = (average hole size) / (average block size), the fraction of memory in holes is **f = k / (k + 2)**

Why? Releasing a block changes the number of holes by -1, 0, 0 or +1, which averages to 0







50% Rule: Example

- Memory size is 30 MB. Average hole size = average block size = 2 KB (1 KB = 1000 bytes). The 50% rule holds.
  - (a) How many holes are there? (b) How many occupied blocks?
  - (c) How much memory is lost in holes?
  - (d) If an average hole is only 1/4 of an average block, what fraction of memory is in holes?



1 hole for every 2 blocks

Answers

Partitions = 30 MB / 2 KB = 15,000

(a) 15,000 / 3 = **5,000 holes** (b) 2 × 5,000 = **10,000 blocks**

(c) 5,000 × 2 KB = **10 MB** (one third of memory)

(d) k = 0.25, f = 0.25 / 2.25 = **1/9 ≈ 11%** (smaller holes waste less)





Swapping and Compaction

- No hole is large enough: **swapping** moves a block to disk for a while; **compaction** shifts blocks to merge holes
- Both need **dynamic relocation**, since a block may come back at a different address

Now: a request of size 9 arrives. Free space = 18, largest hole = 6



Swapping: block 2 goes to disk; the new hole 5 + 2 + 4 = 11 takes the request



Full compaction: move all blocks to one end; one hole of 18



Smart compaction: move only block 2 into the hole of 3; a hole of 11 appears



Compaction is fast (memory to memory) but can never create more than the total free space; swapping uses the slow disk but can repeat until enough space is free





Compaction Cost: Example

- Memory size is 96 MB. Average hole size = average block size. The 50% rule holds.
- Reading or writing one 4-byte word takes 10 ns.
  - (a) How much data must be moved if every block is moved?
  - (b) How long does a full compaction take?
  - (c) Why might the OS stop as soon as a large enough hole exists?

Answers

(a) Blocks fill 2/3 of memory: 2/3 × 96 MB = **64 MB** = 16 million words

(b) Each word is read once and written once: $16 \times 10^6 \times 2 \times 10$ ns = **0.32 s**

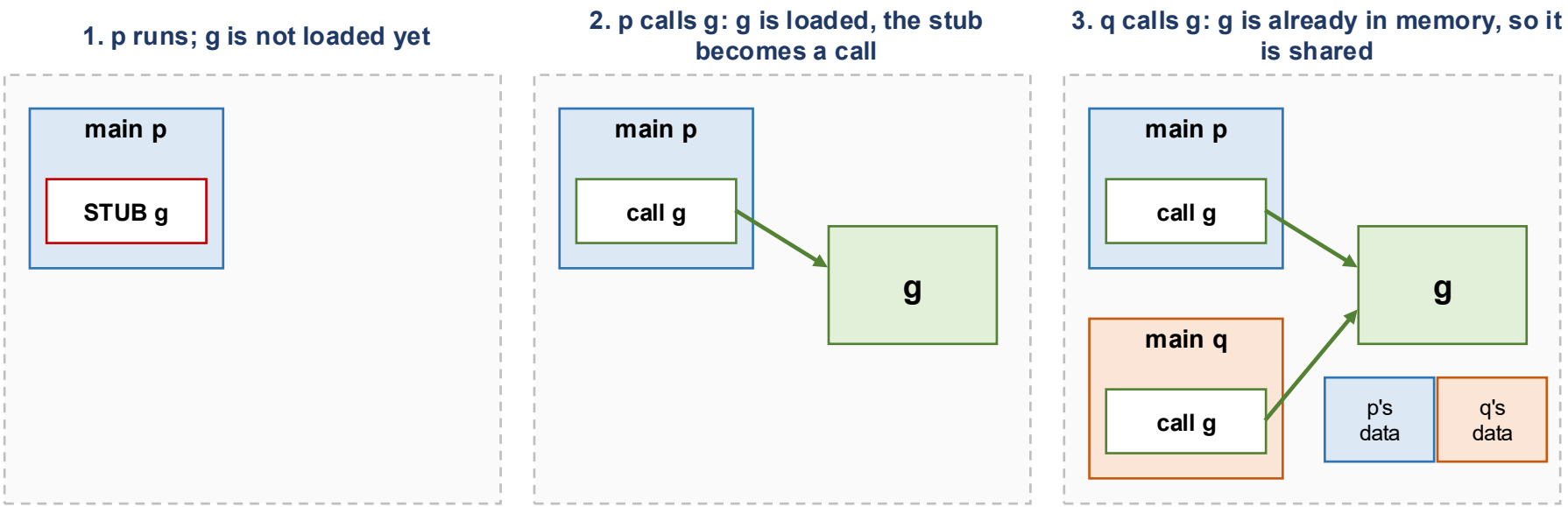
(c) Partial compaction moves fewer blocks, so it takes less time, and the CPU is not doing useful work while memory is being compacted





Dynamic Linking and Sharing

- **Static linking**: all modules are linked and loaded before the run, even ones that are never called
- **Dynamic linking**: each library call is a small **stub**; the module is found and loaded only on its first call (Windows **DLLs**)
- **Sharing**: one copy of a module is linked to many processes; the code is shared, each process keeps its own data







Paging

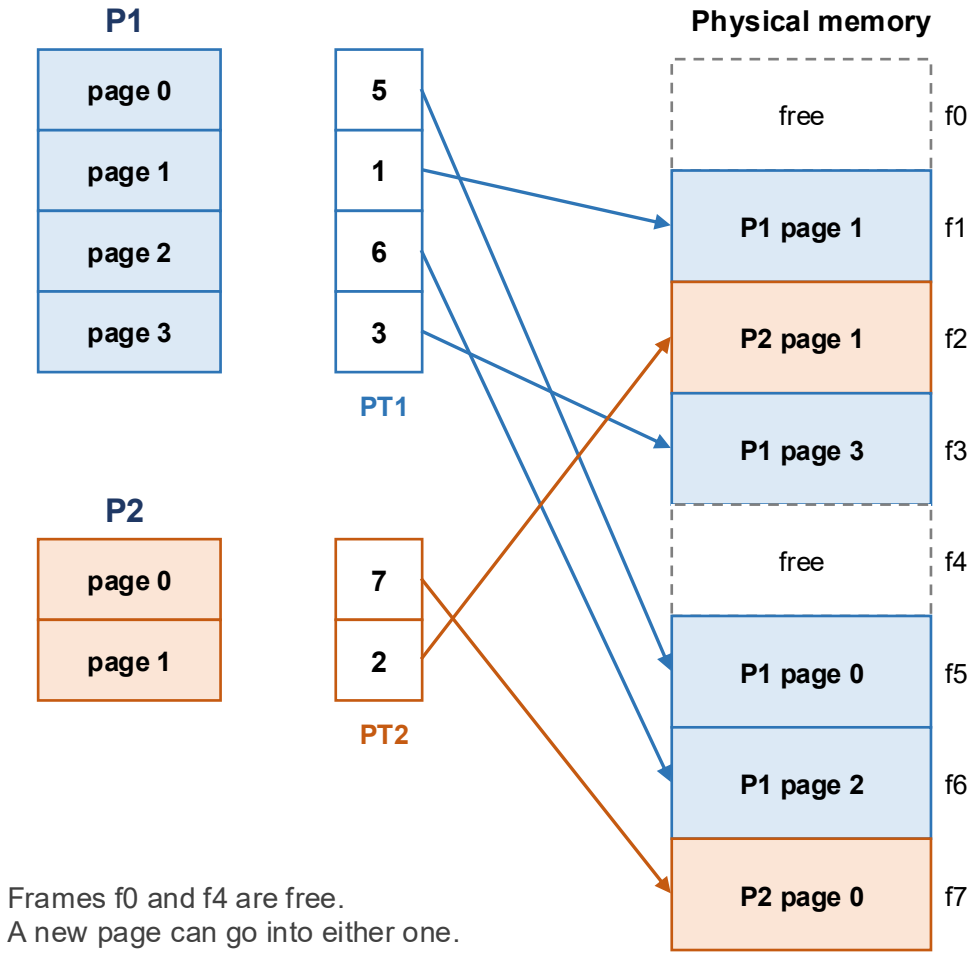
- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
  - Avoids external fragmentation
  - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called **frames**
  - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size ***N*** pages, need to find ***N*** free frames and load program
- Set up a **page table** to translate logical to physical addresses
- Backing store likewise split into pages
- Still have Internal fragmentation





Pages, Frames and Page Tables

- **Page**: a fixed-size block of a logical address space
- **Frame**: a block of physical memory of the same size
- **Page table**: one per process; entry p holds the frame number of page p
- Any page fits in any free frame: **no external fragmentation** and no searching for holes



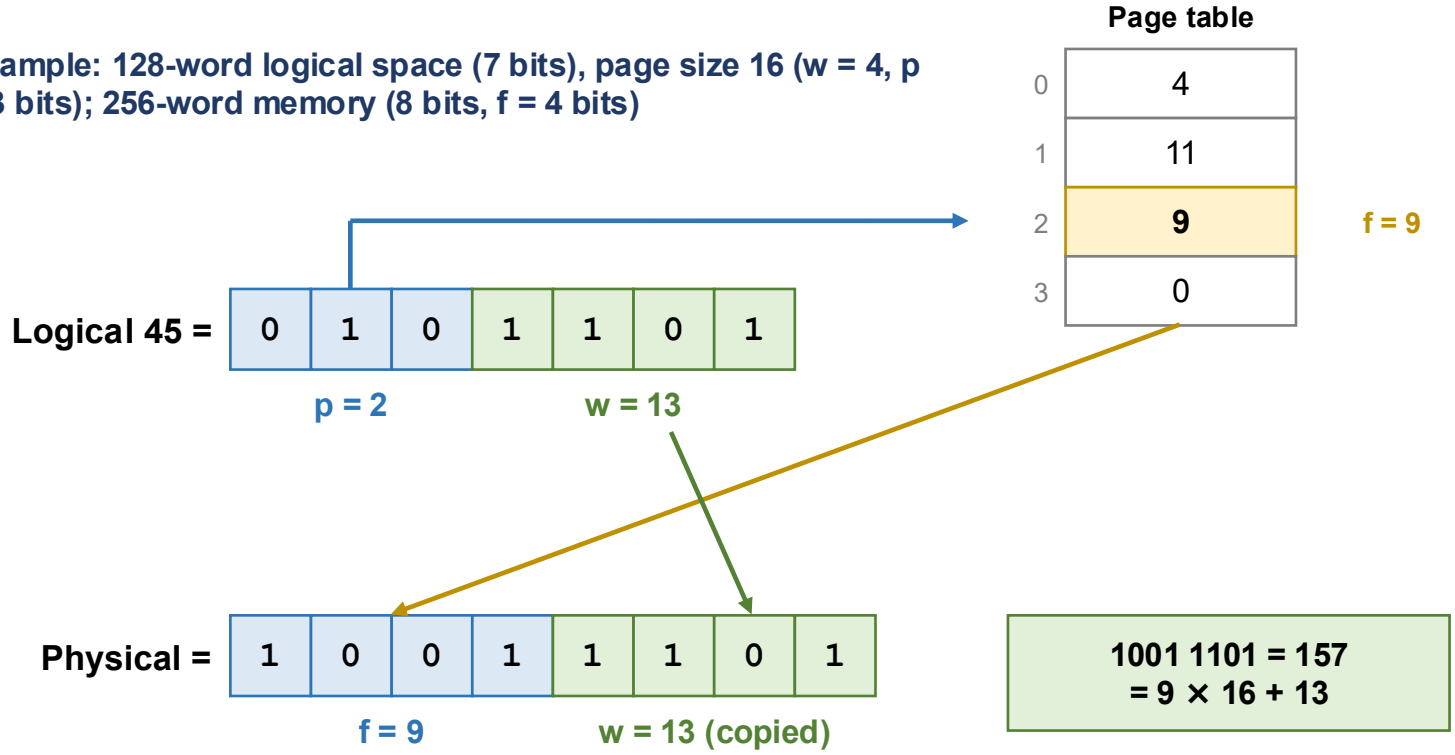




Address Structure in Paging

- A logical address = **page number p** (high bits) followed by **offset w** (low bits)
- With w offset bits a page has 2^w words; with p page bits there are 2^p pages
- A physical address = **frame number f** followed by the same offset w (frame size = page size)

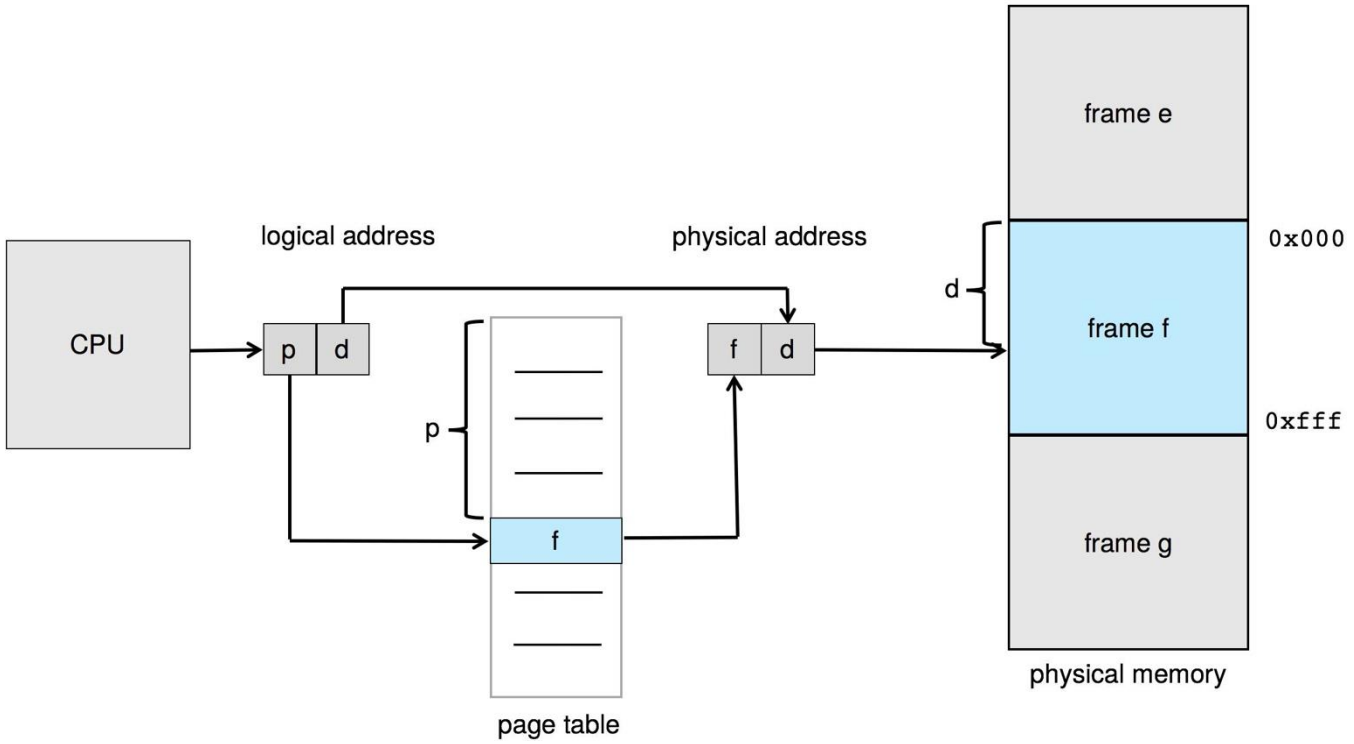
Example: 128-word logical space (7 bits), page size 16 (w = 4, p = 3 bits); 256-word memory (8 bits, f = 4 bits)







Paging Hardware







Paging Translation: Example

- p = LA **div** page size, w = LA **mod** page size, physical address = f × page size + w
- Page size = 1024 words. A dash means the page is not in the program (invalid).

Page table

0	**6**
1	**-**
2	**9**
3	**2**
4	**7**

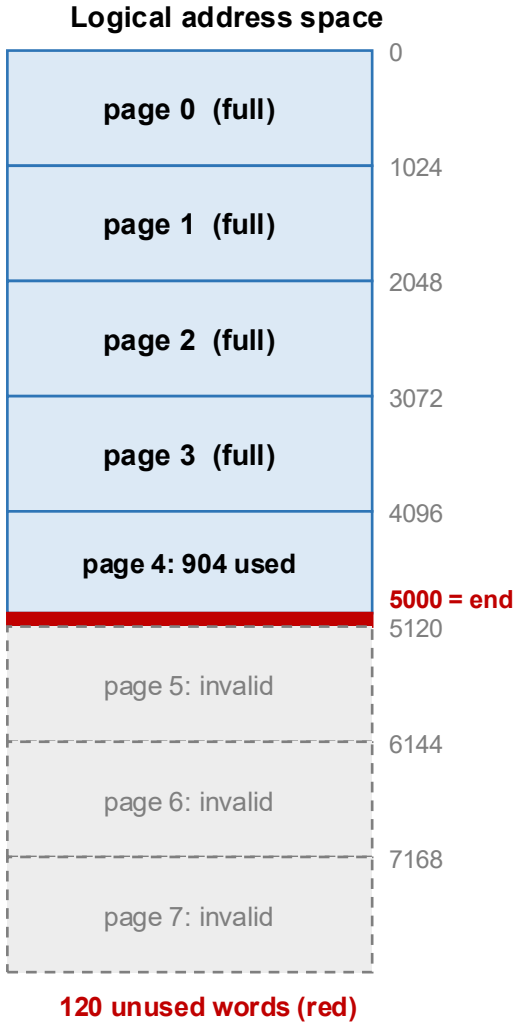
Logical address	p	w	f	Physical address
3100	3	3100 - 3072 = 28	2	2 × 1024 + 28 = 2076
4500	4	4500 - 4096 = 404	7	7 × 1024 + 404 = 7572
1500	1	476	-	**invalid page: trap**
9	0	9	6	6 × 1024 + 9 = 6153

- Only the page number changes; the offset w is copied unchanged
- Hide the table body in class and let students fill it in





Internal Fragmentation



- A program is rarely an exact number of pages, so the end of its last page is wasted: **internal fragmentation**
- Example: page size 1024, program 5000 words
  - 5 pages; 5 × 1024 - 5000 = **120 words** wasted
- Worst case: almost one page; on average, half a page per process
- Smaller pages waste less but need larger page tables
- **Bound check**: pages 5-7 are marked invalid, but address 5050 is in the valid page 4. Each address is also compared with the program size: 5050 ≥ 5000, so it is rejected





Practice: Address Sizes

- (a) A logical address space has 8 pages of 2048 words; memory has 128 frames. How many bits are in a logical address? In a physical address?
- (b) Addresses are 16 bits and a page is 256 words. How many pages are there?
- (c) An address has a 5-bit page number and a 6-bit offset. What is the page size? Write address (3, 10) in binary.

Answers

(a) w = 11 bits ($2048 = 2^{11}$), p = 3 bits (8 pages): logical = **14 bits**;
f = 7 bits (128 frames): physical = 7 + 11 = **18 bits**

(b) w = 8 bits, p = 16 - 8 = 8 bits: 2^8 = **256 pages**

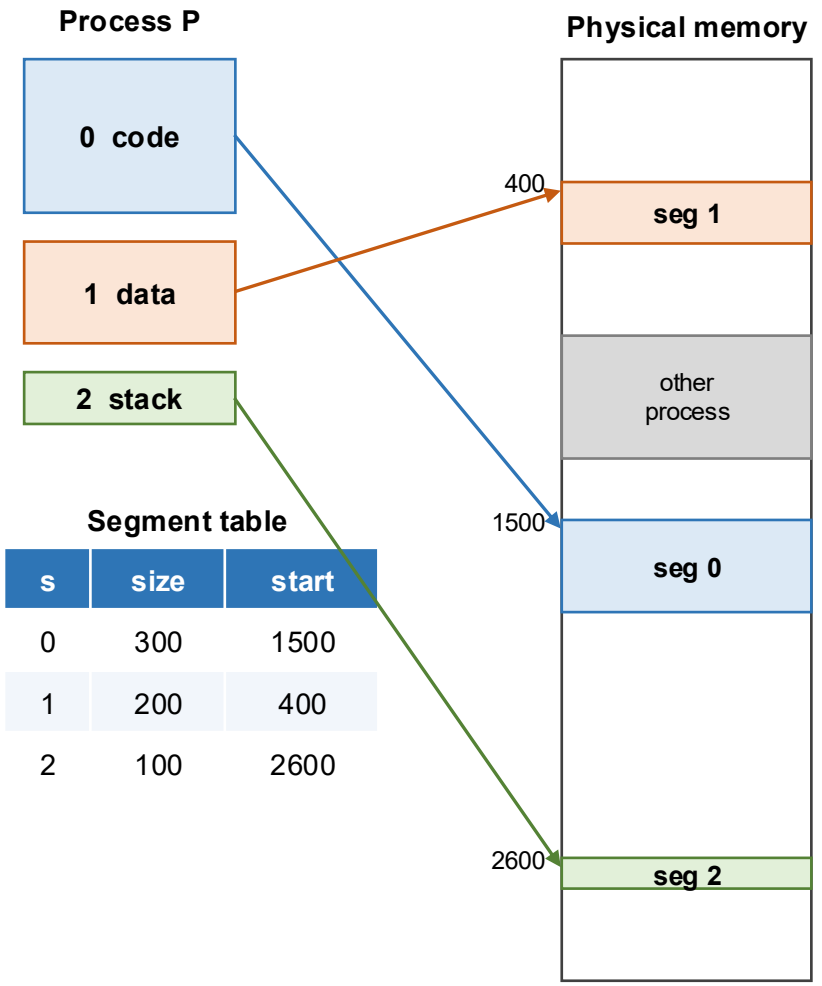
(c) page size 2^6 = **64 words** (32 pages); (3, 10) = `00011 001010`





Segmentation

- With paging, code, data and stack are glued into one space; page edges cut through them, so sharing and protecting a part is awkward
- **Segment**: a variable-size logical block (code, data, stack ...) with its own number s
- **Segment table**: one entry per segment holding its **size** and **start address**
- Pure segmentation: each segment is contiguous in memory, so **external fragmentation** returns



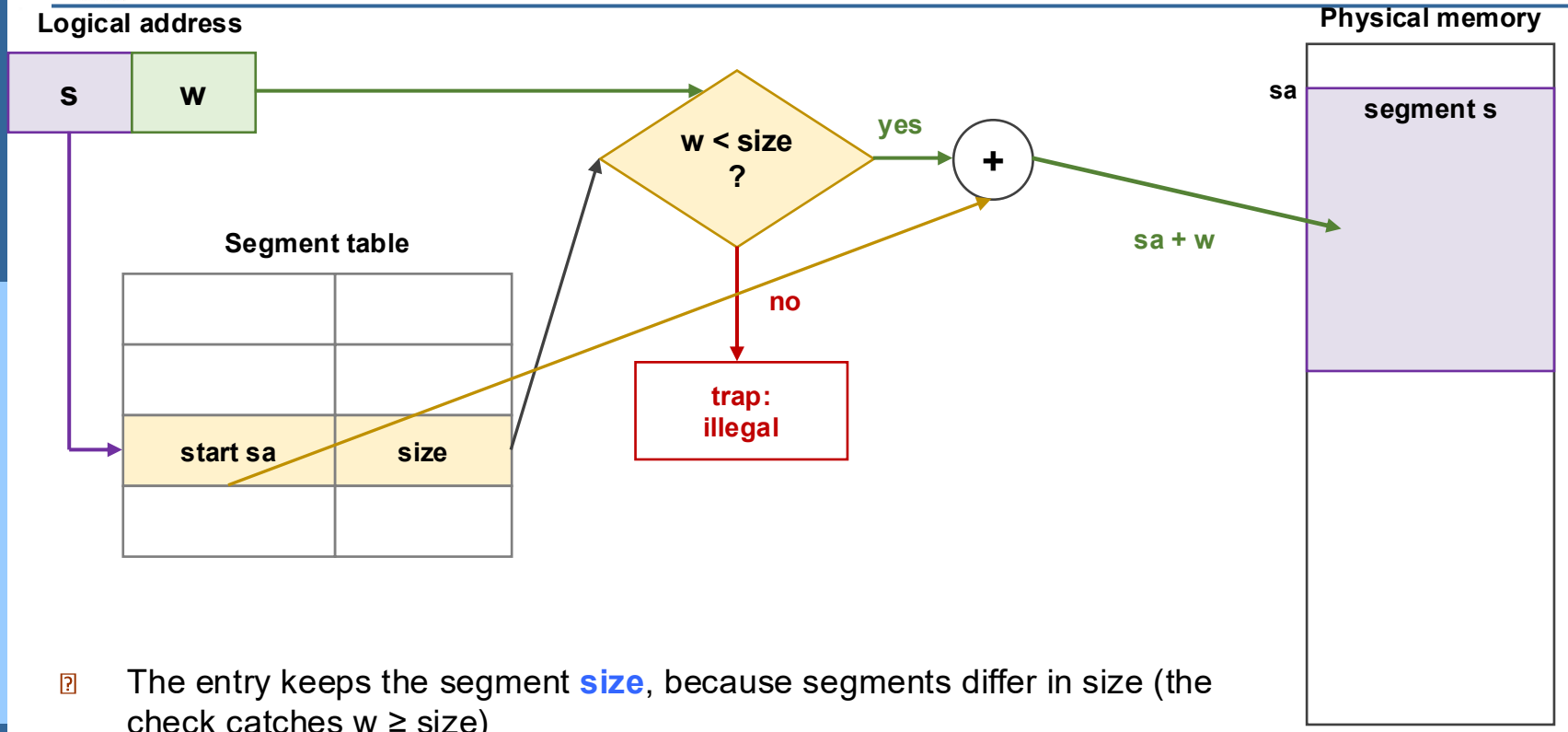
Segment table

s	size	start
0	300	1500
1	200	400
2	100	2600





Segmentation: Address Translation



- The entry keeps the segment **size**, because segments differ in size (the check catches w ≥ size)
- The entry keeps the full **start address**, because there are no fixed-size frames





Segmentation: Example

Segment table

s	size	start
0	600	1400
1	150	300
2	90	2100
3	1000	3000

- Translate each logical address (s, w), or say it is illegal:
  - (0, 320) (1, 160) (2, 89)
  - (3, 0) (4, 5)

Answers

(0, 320): 320 < 600, so PA = 1400 + 320 = **1720**
(1, 160): 160 ≥ 150, **illegal** (offset beyond the segment)
(2, 89): 89 < 90, so PA = 2100 + 89 = **2189** (the last word of segment 2)
(3, 0): PA = **3000**
(4, 5): **illegal**, there is no segment 4





Paging vs. Segmentation

	Paging	Segmentation
Unit size	fixed (page = frame)	variable
Placement	any free frame	a hole large enough
Logical address	(p, w)	(s, w)
Table entry	frame number f	start address + size
Physical address	(f, w), f × page size + w	start + w, if w < size
External fragmentation	no	yes
Internal fragmentation	yes (last page)	no
Sharing / protection of a part	awkward	easy, by segment

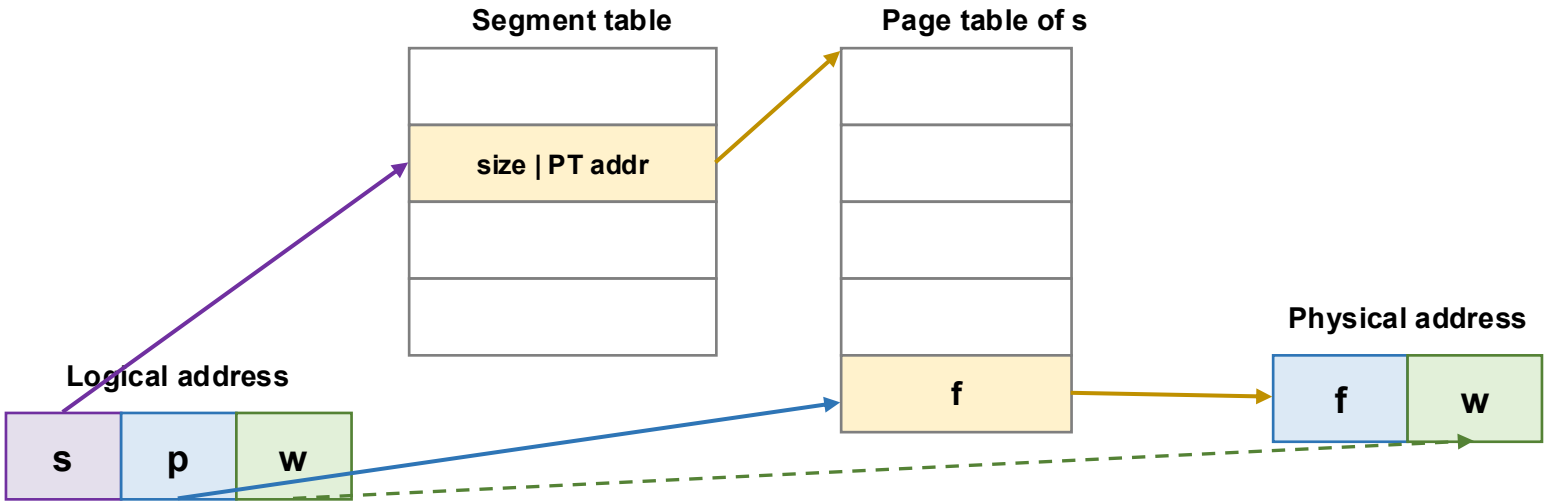
Combining them keeps the best of both: variable segments, each made of fixed pages





Segmentation with Paging

- Each segment is split into pages: a logical address is **(s, p, w)**
- Segment table entry s → size of s and its **page table**; page table entry p → **frame f**; physical address is (f, w)



- Check first: if (p, w) ≥ size of s, the address is illegal
- Without help, every reference needs **3 memory accesses**: segment table, page table, then the word





Seg. with Paging: Example

- Page size 512 words. Segment 1 is 1300 words long; its page table is shown.
  - (a) Translate (1, 2, 100)
  - (b) Translate (1, 2, 400)
  - (c) Addresses have 32 bits and there can be 2^{12} segments. How many bits are p and w? How many pages can a segment have?

Page table of segment 1

0	**6**
1	**3**
2	**11**

Answers

(a) (p, w) = 2 × 512 + 100 = 1124 < 1300, OK; f = 11, PA = 11 × 512 + 100 = **5732**

(b) (p, w) = 2 × 512 + 400 = 1424 ≥ 1300: **illegal**, although page 2 exists

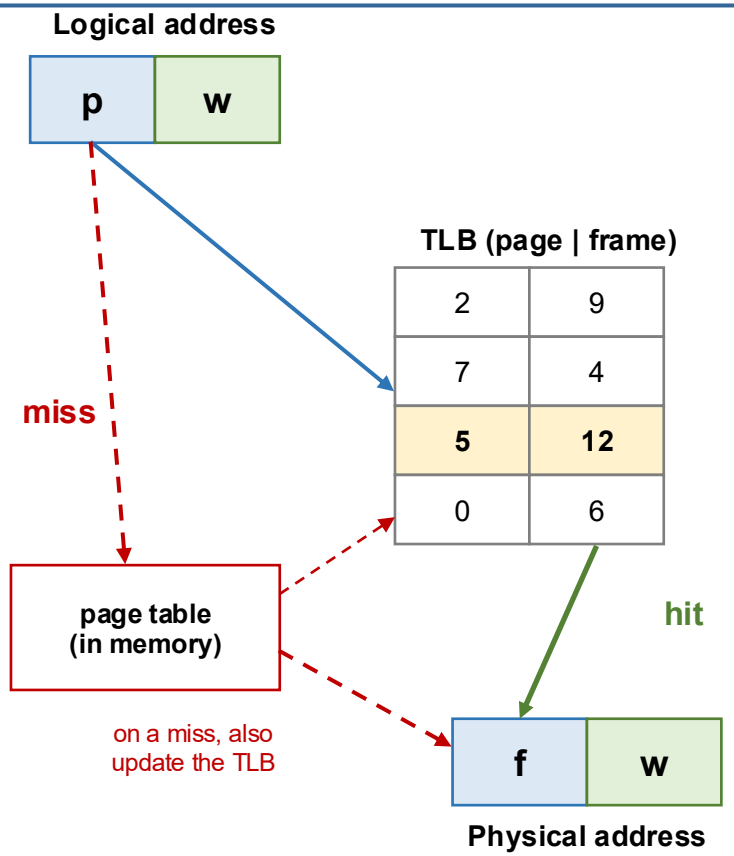
(c) w = 9 bits ($512 = 2^9$), s = 12 bits, so p = 32 - 12 - 9 = **11 bits**:
up to 2^{11} = **2048 pages** per segment





Translation Look-Aside Buffer

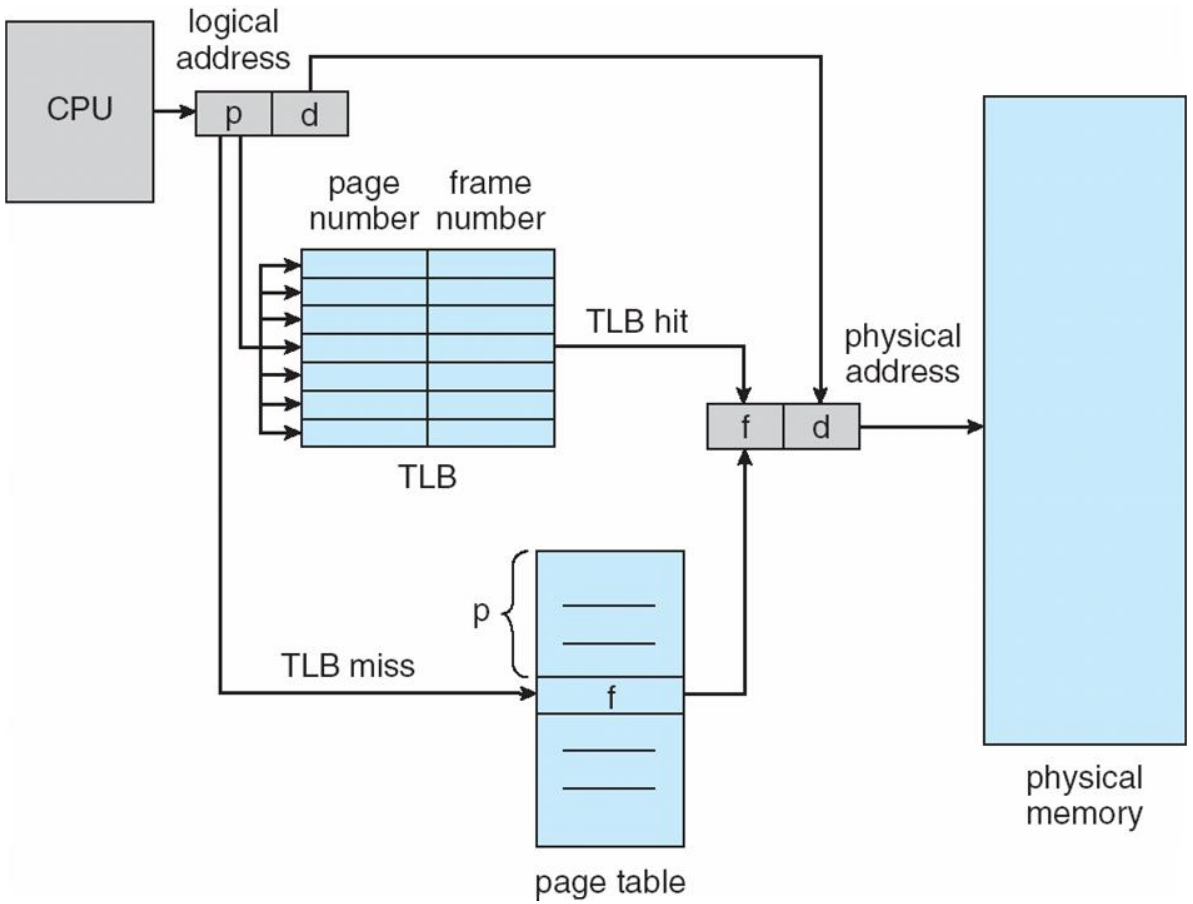
- Paging doubles the memory accesses, segmentation with paging triples them
- **TLB**: a small, fast associative memory holding recent translations (page → frame)
- **Hit**: frame found in the TLB, no table access
- **Miss**: use the tables, then save the pair, replacing the **least recently used** entry
- Works because of **locality**: recently used pages are likely to be used again







Paging Hardware With TLB







Effective Access Time (EAT)

- **Hit ratio h**: the fraction of references whose translation is in the TLB
- Paging example: memory access 50 ns, TLB lookup 5 ns
  - Hit: 5 + 50 = 55 ns Miss: 5 + 50 (page table) + 50 = 105 ns
  - EAT = h × 55 + (1 - h) × 105

Hit ratio h	EAT	vs. 100 ns without a TLB
0.90	0.9 × 55 + 0.1 × 105 = 60 ns	40% faster
0.98	0.98 × 55 + 0.02 × 105 = 56 ns	44% faster

- Practice: with segmentation and paging (3 accesses, 150 ns without a TLB), which h brings the time down to 90 ns?

Answers

Hit: 5 + 50 = 55 ns, miss: 5 + 150 = 155 ns; h × 55 + (1 - h) × 155 = 90 → **h = 0.65**





Practice: TLB Hits and Misses

A TLB has 3 entries, starts empty and replaces the least recently used entry. Which references hit? What is left in the TLB at the end?

#	Address (s, p, w)	Hit or miss	TLB after (s, p) pairs, LRU first
1	(2, 0, 7)	miss	(2,0)
2	(1, 3, 0)	miss	(2,0) (1,3)
3	(2, 0, 40)	**hit**	(1,3) (2,0)
4	(4, 1, 5)	miss	(1,3) (2,0) (4,1)
5	(3, 0, 0)	miss, evict (1,3)	(2,0) (4,1) (3,0)
6	(1, 3, 9)	miss, evict (2,0)	(4,1) (3,0) (1,3)

- Only reference 3 hits: the offset w does not matter, only (s, p)
- The green columns are the answers; hide them in class





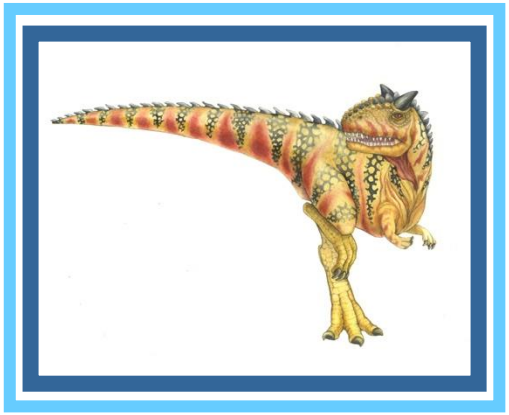
Summary

Technique	Unit placed in memory	Mapping kept in	External frag.	Internal frag.	Accesses per reference
Contiguous	whole program	relocation + limit registers	yes	no	1
Paging	fixed-size pages	page table	no	yes	2
Segmentation	variable segments	segment table	yes	no	2
Segmentation + paging	pages of segments	segment + page tables	no	yes	3

- Dynamic relocation lets the OS move programs (swapping, compaction)
- First/next/best/worst-fit choose holes; the 50% rule says 1/3 of partitions are holes
- A TLB hit cuts translation to one TLB lookup plus one memory access



End of Chapter 9



Silberschatz, Galvin and Gagne ©2018